

تقنية الـ **JIT (Just-In-Time) Compilation** تقوم بترجمة الكود الوسيط (IL/MSIL) إلى **Native Machine Code** لحظة التشغيل. (Runtime) هذه العملية تسبب بطئاً في أول مرة يتم فيها تشغيل التطبيق (Startup Overhead).

ولتقليل هذا الـ Overhead ، توجد عدة حلول وتقنيات:

١. **AOT Compilation (Ahead-Of-Time):** ترجمة كود الـ IL بالكامل إلى Native Machine Code أثناء بناء المشروع (Build Time) وقبل التشغيل، مما يلغي الحاجة للـ JITting أثناء الـ Runtime تماماً وتحسن سرعة بدء التشغيل بشكل كبير.
٢. **ReadyToRun (R2R) Images:** صيغة تمزج بين AOT و JIT ؛ حيث يتم تجهيز وتوليد Native Code لأغلب أجزاء التطبيق مسبقاً، مع ترك بعض الأجزاء المتقدمة ليتعامل معها الـ JIT أثناء التشغيل.
٣. **Tiered Compilation (الترجمة المترتبة:)**
 - **Tier 0 (Quick JIT):** يقوم المحرك بترجمة الكود بسرعة وبدون إجراء تحسينات (Optimizations) معقدة ليعمل البرنامج فوراً.
 - **Tier 1 (Optimized JIT):** يراقب المحرك الأكواد التي تتكرر كثيراً (Hot Methods) ويقوم بإعادة ترجمتها وتحسينها في الخلفية لتعطي أفضل أداء أثناء التشغيل المستمر.
٤. **NGen (Native Image Generator):** أداة قديمة في .NET Framework. التقليدي تقوم بتحويل ملفات الـ Assemblies إلى Native Images وتخزينها في الـ Native Image Cache على جهاز المستخدم قبل تشغيل التطبيق.